

Bennett Moore

Prof. Shen

CPS 352

3/26/25

### Approach:

- The basic idea for the K-means algorithm, as described in class, is pretty simple:
  - Convert the 2D grid of pixels to a 1D array (flattening), because the 1D array is easier to work with
  - Do the following steps in a loop...
  - Compare each pixel to each centroid. Figure out the centroid each pixel is closest to, which determines its cluster; save the cluster ID in an array
  - Regenerate the centroids by calculating the mean of all the pixels in each cluster
  - Check to see if the centroids have stabilized. Are pixels still moving between clusters enough that the centroids keep changing? If so, continue looping...but if the pixels are staying in the same spots, exit the loop
  - Reconstruct the compressed version of the image by iterating through the array of cluster IDs and assigning each pixel to be the centroid of its cluster
  - Reshape the 1D array back into a 2D array of pixels with the dimensions of the original image
- As for the rest of the project:
  - Creating the dictionary was pretty simple. It's just a cache of compressed images. Each time I generated a new one, I saved it with the key being the number of clusters
  - I cleaned up some things in the framework, including making use of a main() function, which I consider to be good practice
  - The ImageCompressor class handles opening the image and converting it into a 1D array of pixels and back

### Challenges:

- I implemented the algorithm using basic Python loops at first, but this was super inefficient because iterating through hundreds of pixels over and over again takes a long time, especially if you have to iterate through lots of clusters for each pixel
- So, I got the help of ChatGPT and created a version of the algorithm using NumPy's vectorized looping functions, which run on precompiled C code under the hood...this is of course orders of magnitude more efficient than the Python loops
- It was also difficult when writing the first algorithm to figure out a way to reconstruct the image at the end, at least in a semi-performant way. I had the groups of pixels in

each cluster, but I didn't know where in the image they belonged. I considered searching the original list of pixels for each clustered pixel and replacing it with the centroid, but decided that would be far too inefficient, and opted for a new data structure (the cluster ID list) instead

### Screenshots:



